Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project

Author: Qihaohan Zheng

Date: October 1, 2026

Abstract

This project investigates whether machine learning models can identify the large language model (LLM) family that generated a response. A balanced dataset was constructed using 300 shared English prompts and 900 responses from Claude, DeepSeek, and GPT. The prompts cover factual knowledge, explanations, mathematics, coding, writing, and summarization. A convolutional neural network (CNN) and a bidirectional long short-term memory network (LSTM) were evaluated using prompt-grouped training, validation, and test sets.

For output-only classification, the CNN achieved 65.19% accuracy and a macro-F1 score of 0.6476, while the LSTM achieved 51.85% accuracy and a macro-F1 score of 0.4930. Both input-only classifiers achieved 33.33% accuracy, and adding prompts to responses did not improve performance. In a separate cross-domain experiment, CNN accuracy decreased from 69.57% on coding, mathematics, and factual tasks to 48.67% on writing. Descriptive analysis revealed differences in response length, lexical diversity, and formatting. However, collection-format artifacts, limited vocabulary coverage, truncation, and overfitting complicate interpretation. The results support dataset-specific LLM attribution, while leaving the stability of general LLM fingerprints unresolved.

1. Introduction and Research Questions

LLMs can answer questions, generate code, explain concepts, and produce creative writing. Although their responses may contain similar information, different models may express that information differently. Response length, word choice, punctuation, and formatting could provide clues about the source model.

This project treats LLM attribution as a three-class classification problem. Given text associated with a response, the classifier predicts whether its recorded source is Claude, DeepSeek, or GPT. The objective is to investigate whether source information can be learned, which input conditions are useful, and whether learned patterns transfer to an unfamiliar task.

The study addresses four research questions:

Research question	Investigation
RQ1	Can CNN and LSTM classifiers identify an LLM from its response?
RQ2	Does adding the user’s prompt improve classification?
RQ3	Do learned patterns generalize across tasks or domains?
RQ4	What linguistic and structural characteristics distinguish the collected responses?

This is a closed-set attribution task. It does not distinguish human writing from AI writing, evaluate answer correctness, or identify model families that were absent from training.

2. Dataset Construction

2.1 Prompt Design

The dataset contains 300 English prompts, numbered P001–P300. The prompt bank was designed with LLM assistance and divided into six categories.

Table 1. Dataset composition

Category	Prompts	Responses
Factual	50	150
Explanation	50	150
Math	50	150
Coding	50	150
Writing	50	150
Summarization	50	150
Total	**300**	**900**

Each category contains 20 prompts labeled Easy, 20 Medium, and 10 Hard. These labels describe the intended design and were not empirically validated.

The same prompts were used for all three LLM families. This controls for question differences that could otherwise allow a classifier to associate particular topics with particular models. Each family contributes 300 responses, resulting in balanced classes.

2.2 Data Records and Validation

The merged dataset includes:

response_id, prompt_id, LLM_name, model_version, LLM_Input, LLM_output, category, task_type, and difficulty.

The merge workflow checked for missing prompt IDs, duplicate prompt-model pairs, empty responses, and disagreement between the recorded input and the reference prompt. Original response text was retained, including code and formatting.

The supplied records identify source families, but the model-version fields are blank. Exact generation settings, collection times, and conversation or batch boundaries were not preserved in the reviewed records. Therefore, this study analyzes the recorded families without claiming verified version-level attribution. Answer correctness and compliance with requested response lengths were not systematically evaluated.

2.3 Training, Validation, and Test Sets

The main dataset was divided by prompt_id, using seed 42 and approximately balanced category allocations.

Table 2. Main dataset partitions

Partition	Unique prompts	Responses	Responses per LLM
Training	210	630	210
Validation	45	135	45
Test	45	135	45

All three responses to a prompt remain in the same partition. No prompt ID overlaps across partitions. This prevents exact-question overlap, although related topics or similar questions may still occur in different sets.

3. Preprocessing and Model Implementation

3.1 Text Preprocessing

The tokenizer preserves capitalization, punctuation, and newline tokens. It extracts alphanumeric sequences, underscores, and other non-whitespace characters. Actual line endings are standardized, but literal escaped newline strings remain in classification input.

Vocabularies are fitted using training text only. Padding uses token ID 0, and unknown tokens use ID 1. Short sequences are padded on the right, while long sequences are truncated.

For the main output-only experiment, the vocabulary contains 5,153 entries and the sequence length is 160 tokens. The length was selected from the training-set 95th percentile, subject to a maximum of 512.

Input-only uses prompt text. Input + Output concatenates separately truncated prompt and response sequences with a separator token. Its vocabulary includes both fields from training data. Consequently, input conditions differ in vocabulary and sequence length as well as text content.

3.2 CNN Architecture

The CNN uses:

- A learned 128-dimensional embedding.
- Three convolution branches with kernel sizes 3, 5, and 7.
- Sixty-four filters per branch.
- ReLU activation and maximum pooling.
- A concatenated 192-dimensional representation.
- Dropout of 0.5 and a linear layer producing three logits.

Pooling is restricted to positions centered on actual tokens. The architecture can learn local combinations of words and formatting tokens.

3.3 LSTM Architecture

The recurrent model uses:

- A learned 128-dimensional embedding.
- One bidirectional LSTM layer.

- Sixty-four hidden units in each direction.
- A concatenated 128-dimensional final representation.
- Dropout of 0.5 and a linear classification layer.

Packed sequences use actual lengths so that padding does not become part of the recurrent representation.

3.4 Training and Evaluation

Table 3. Training configuration

Setting	Value
Loss function	Cross-entropy
Optimizer	Adam
Learning rate	0.001
Weight decay	0.0001
Batch size	32
Maximum epochs	30
Early-stopping patience	5 epochs
Gradient clipping	Maximum norm 1.0
Random seed	42
Checkpoint selection	Minimum validation loss
Recorded framework	PyTorch 2.14.1+cpu

Accuracy and macro-F1 are the primary metrics. Precision, recall, confusion matrices, and training histories provide additional information. Macro-F1 gives equal importance to each source family.

The balanced three-class random-guessing baseline has expected accuracy of 33.33%. Experiments use a single training seed; statistical significance and confidence intervals were not evaluated.

4. RQ1: Identifying LLMs from Responses

Table 4. Output-only test results

Model	Selected epoch	Accuracy	Macro-F1
CNN	8	65.19%	0.6476
LSTM	7	51.85%	0.4930

The CNN correctly classified 88 of 135 responses, compared with 70 for the LSTM. Both exceeded the random-guessing baseline. The CNN advantage was 13.33 percentage points.

Table 5. Class-level output-only results

LLM	CNN Precision	CNN Recall	CNN F1	LSTM Recall
Claude	0.67	0.53	0.59	0.22
DeepSeek	0.62	0.80	0.70	0.69
GPT	0.68	0.62	0.65	0.64

DeepSeek was the strongest CNN class according to F1. The LSTM had particularly low recall for Claude.

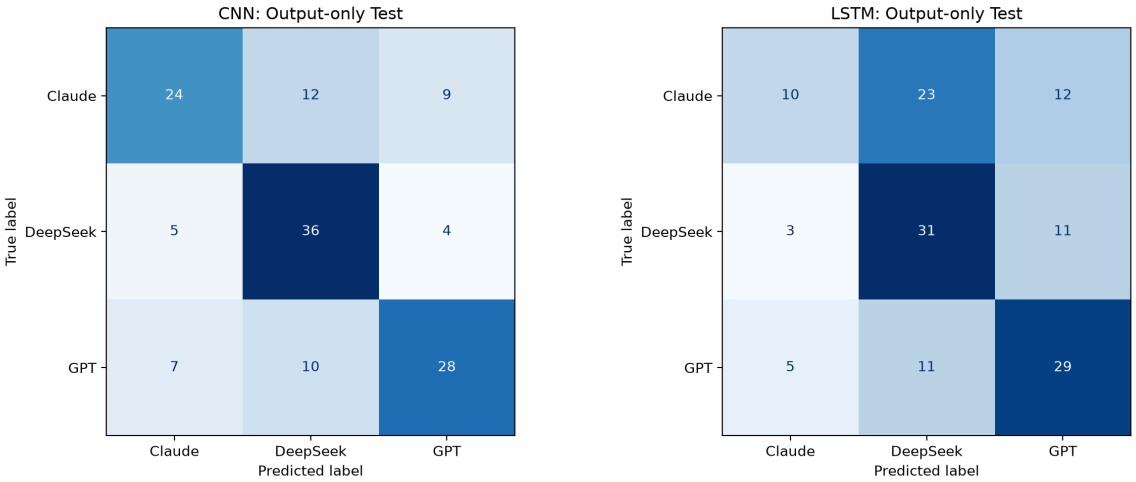


Figure 1. Confusion matrices for output-only attribution on the main test set. Left: CNN. Right: LSTM.

Both models showed overfitting. CNN training accuracy reached 93.49% at epoch 13, while validation accuracy was 65.19%. LSTM training accuracy reached 92.86% at epoch 12, while validation accuracy was 55.56%. The lowest validation losses occurred earlier, at epochs 8 and 7, respectively.

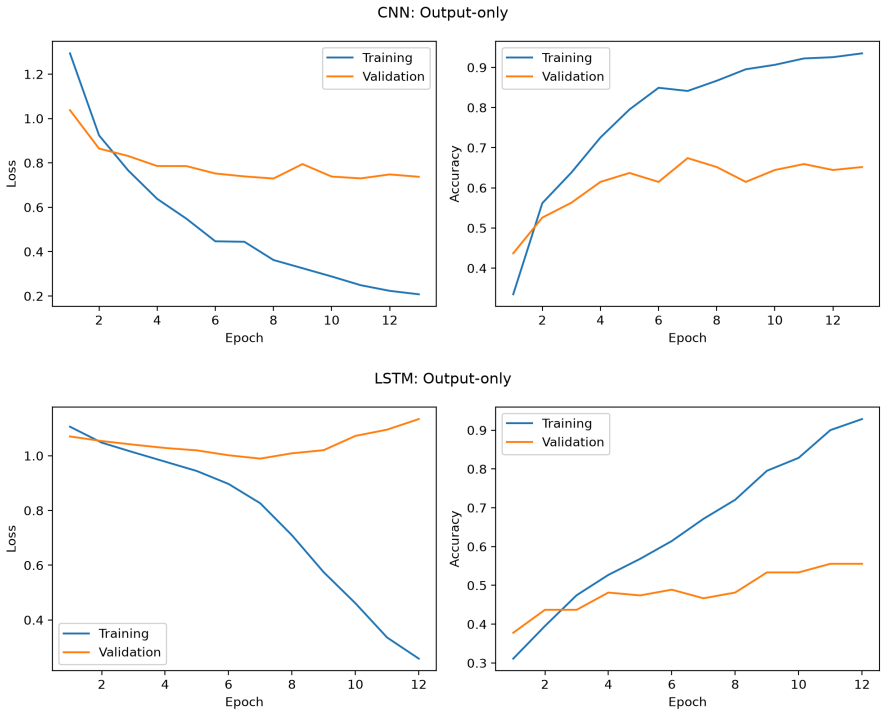


Figure 2. Training and validation loss and accuracy for the output-only models. Top: CNN. Bottom: LSTM.

The results show useful attribution information in the collected responses. However, they do not establish whether the models learned linguistic style, formatting conventions, or collection artifacts.

5. RQ2: Does the Prompt Help?

Table 6. Comparison of input conditions

Model	Input condition	Accuracy	Macro-F1
CNN	Input-only	33.33%	0.3043
CNN	Output-only	65.19%	0.6476
CNN	Input + Output	61.48%	0.6092
LSTM	Input-only	33.33%	0.3222
LSTM	Output-only	51.85%	0.4930
LSTM	Input + Output	43.70%	0.4314

5.1 Input-Only Classification

The input-only result follows directly from the dataset design. Each identical prompt occurs under all three labels. A deterministic classifier makes the same prediction for those three records and therefore gets exactly one correct.

Thus, 33.33% accuracy is expected and does not indicate useful source information in the prompts. Different macro-F1 scores reflect different prediction distributions.

5.2 Input + Output Classification

Adding prompts reduced CNN accuracy by 3.70 percentage points and LSTM accuracy by 8.15 points. These changes correspond to five and eleven fewer correct predictions, respectively.

Common question text may add irrelevant information, and longer sequences may make response patterns harder to learn. However, vocabulary and sequence budgets also differ between conditions. The decline cannot be attributed exclusively to prompt inclusion.

Output-only was the strongest condition for both architectures in this run. The experiment supports the conclusion that responses provide the main discriminative information in this dataset.

6. RQ3: Cross-Domain Generalization

6.1 Experimental Design

New classifiers were trained only on Coding, Math, and Factual responses:

- Training: 105 prompts and 315 responses.
- Validation: 22 prompts and 66 responses.
- In-domain testing: 23 prompts and 69 responses.
- Cross-domain testing: all 50 Writing prompts and 150 responses.

Writing did not participate in vocabulary fitting, length selection, training, or checkpoint selection. The same selected checkpoint was tested in both conditions.

Some Writing records had been used in earlier mixed-domain experiments. They are unseen by the new RQ3 models, but are not a completely untouched corpus at the project level.

6.2 Overall Performance

Table 7. Cross-domain results

Model	In-domain Accuracy	Writing Accuracy	Change	In-domain Macro-F1	Writing Macro-F1
CNN	69.57%	48.67%	−20.90 pp	0.7001	0.4924
LSTM	57.97%	38.67%	−19.30 pp	0.5823	0.3461

Both models performed substantially worse on Writing. The CNN remained stronger, but neither maintained its source-domain performance.

6.3 Vocabulary Coverage and Truncation

The source-training vocabulary contains 2,234 entries, and the selected sequence length is 130 tokens.

Table 8. RQ3 preprocessing differences

Measure	In-domain test	Writing test
Unknown-token rate	14.66%	36.56%
Truncated responses	0 of 69	60 of 150

Writing introduces more unfamiliar vocabulary, and 40% of its responses are truncated. These factors may contribute to the performance decrease alongside changes in task-specific expression.

6.4 Class-Level Results

Table 9. Attribution on Writing

LLM	CNN Precision	CNN Recall	CNN F1	LSTM Precision	LSTM Recall	LSTM F1
Claude	0.35	0.40	0.37	0.33	0.78	0.47
DeepSeek	0.67	0.52	0.58	0.59	0.20	0.30
GPT	0.50	0.54	0.52	0.56	0.18	0.27

The CNN achieved its highest Writing F1 for DeepSeek. The LSTM predicted Claude for 117 of 150 responses, including 39 actual Claude, 39 DeepSeek, and 39 GPT responses. Its high Claude recall therefore accompanied low precision and a strong prediction bias.

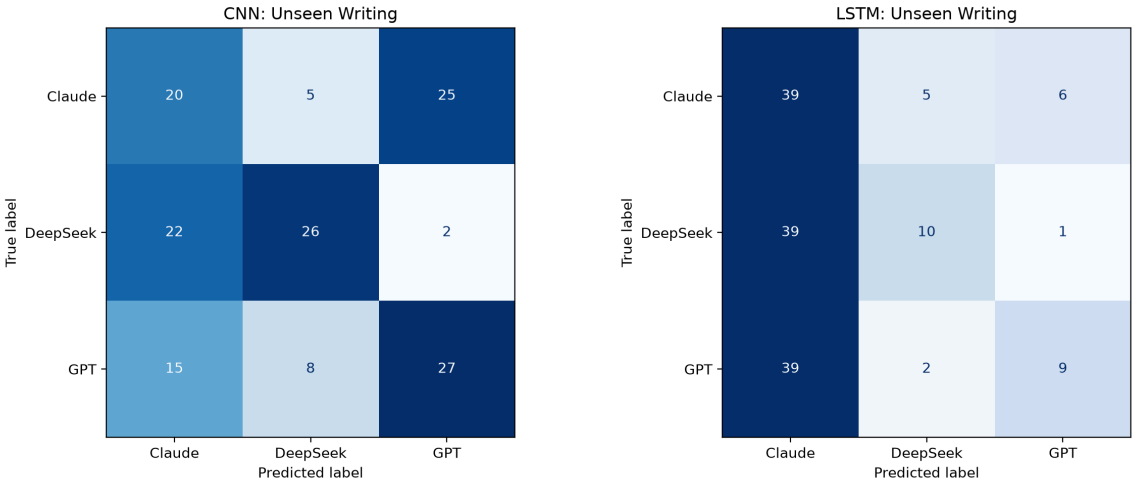


Figure 3. Attribution on the unseen Writing domain. Left: CNN. Right: LSTM.

RQ3 indicates limited cross-domain generalization under the current setup. It does not prove that general LLM fingerprints are absent, because domain changes, vocabulary coverage, truncation, and formatting artifacts are not isolated.

7. RQ4: Linguistic and Structural Characteristics

7.1 Feature Definitions

Descriptive analysis was conducted after training on all 900 responses. Features included character count, English word count, lexical diversity, and the presence of lists, Markdown headings, and code fences.

Lexical diversity was measured using MATTR-50: the average proportion of unique lowercased words within sliding windows of 50 words. Fenced code was removed before this calculation. Responses with fewer than 50 remaining words were excluded.

Lists were detected using line-start bullet or numbered markers, headings using Markdown # markers, and code fences using triple backticks. These definitions do not capture all possible formatting styles.

7.2 Results

Table 10. Descriptive response features

Feature	Claude	DeepSeek	GPT
Mean characters	357.8	196.2	355.6
Mean English words	57.4	31.2	54.8
Mean MATTR-50	0.773	0.792	0.821
MATTR-50 eligible responses	153	43	123
Responses with lists	2.7%	1.7%	3.7%
Responses with Markdown headings	0%	1.0%	0%
Responses with code fences	0%	0%	17.0%

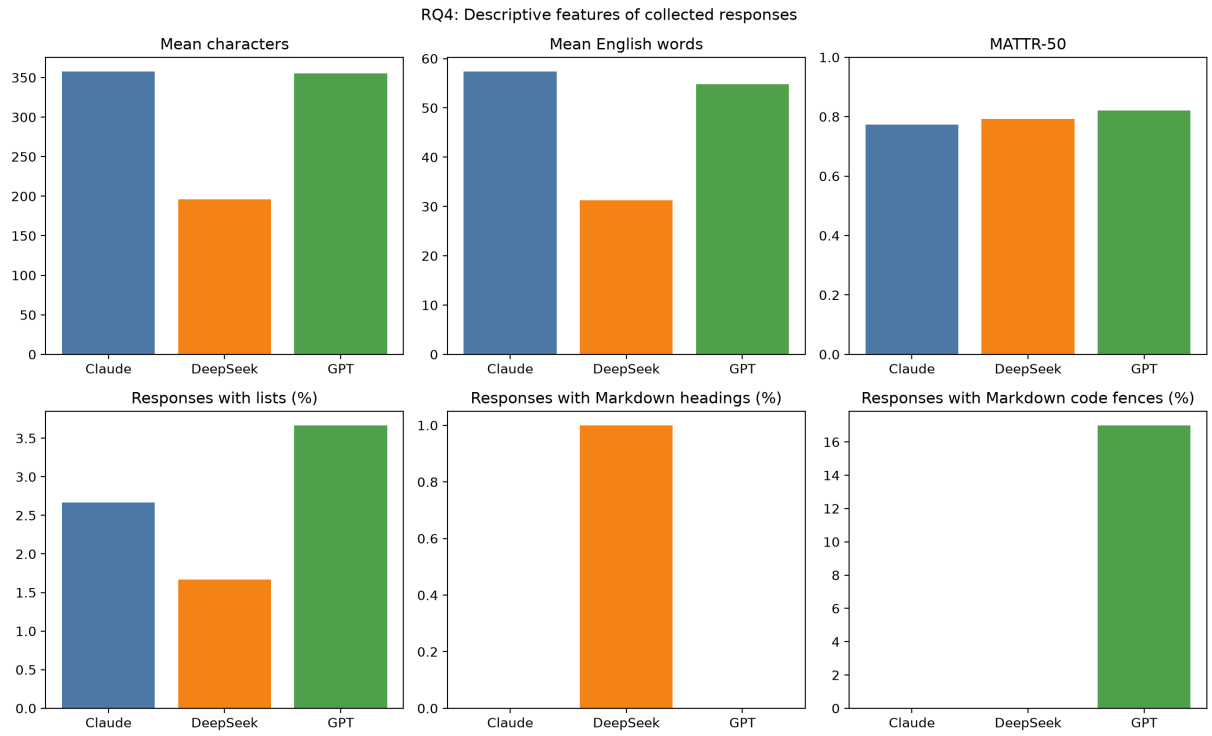


Figure 4. Length, lexical diversity, and formatting features of the collected responses.

DeepSeek responses were considerably shorter. GPT and Claude had similar average lengths, but GPT more frequently used code fences. No code fences were detected for Claude or DeepSeek; this does not mean their responses contained no code.

The MATTR averages use unequal subsets and should not be treated as a fair overall ranking. Unfenced code can also affect the measure.

7.3 Matched-Prompt Comparisons

Comparisons using the same prompts reduce differences caused by question selection.

Table 11. Paired descriptive differences

Comparison	Mean word difference	Mean MATTR-50 difference	Eligible MATTR pairs
GPT − Claude	−2.64	+0.0278	118
GPT − DeepSeek	+23.53	+0.0318	39
Claude − DeepSeek	+26.16	+0.0249	43

These results reinforce the length difference between DeepSeek and the other families. Diversity comparisons remain restricted to jointly eligible responses, and no significance tests were conducted.

7.4 Collection-Format Artifact

Sixty-three DeepSeek responses contained literal newline or tab escape patterns. No such patterns were flagged for Claude or GPT. RQ4 heuristically restored these escapes for analysis, while the classification experiments retained them.

This creates a possible source-associated shortcut. A classifier may recognize how answers were saved rather than only how the LLM expressed them. Restoration is also imperfect because some escapes may be intentional code.

Length and formatting differences provide plausible explanations for attribution performance, but no ablation experiment established that the networks actually relied on these features.

8. Discussion and Limitations

The results demonstrate that the recorded source families can be distinguished above the expected guessing baseline within this dataset. The CNN performed better than the LSTM in all response-containing main conditions and in cross-domain evaluation.

Several limitations constrain interpretation:

1. **Collection metadata:** Exact model versions and generation settings were not recorded.
2. **Serialization differences:** Literal escapes may provide a model-associated shortcut.
3. **Small dataset and one seed:** Results may vary with training initialization and different test samples.
4. **Overfitting:** Training accuracy rose substantially above validation accuracy.
5. **Preprocessing differences:** Input conditions use different vocabularies and sequence budgets.
6. **Cross-domain representation loss:** Writing has more unknown tokens and truncated responses.
7. **Limited feature evidence:** Descriptive statistics do not prove what the networks learned.
8. **Incomplete response auditing:** Correctness, instruction compliance, and collection-time truncation were not systematically checked.

Future work should use verified model versions, consistent independent collection, multiple seeds, and fresh held-out responses. Normalized-versus-original experiments could assess serialization effects. Length-only baselines and controlled formatting ablations could test whether simple structural signals explain the classification results.

9. Lessons and Experience

This project showed that data design and inspection are as important as model implementation. Keeping shared prompts together prevents exact-question overlap, while fitting vocabularies only on training data prevents preprocessing leakage. The input-only experiment provides a useful control because its expected accuracy follows from the balanced prompt design.

Training histories demonstrated that high training accuracy does not necessarily indicate successful generalization. Validation-based checkpoint selection was necessary because both

models continued improving on training data after validation loss stopped improving.

The most important data-quality lesson was the discovery of literal escape patterns associated with one source family. This finding changed the interpretation of the results: attribution performance may reflect collection artifacts as well as language style. Future experiments should document collection settings more carefully and test these shortcuts directly.

10. Conclusion

RQ1 demonstrated above-baseline attribution, with an output-only CNN reaching 65.19% accuracy. RQ2 showed that adding prompts did not improve performance, while input-only accuracy was exactly 33.33%. RQ3 revealed weaker transfer to Writing, with CNN and LSTM accuracies of 48.67% and 38.67%. RQ4 identified differences in response length and formatting, together with a collection-format artifact.

The findings support limited attribution of the collected responses. They do not yet establish stable, intrinsic fingerprints that generalize across domains, model versions, or collection methods.

Sources and Assistance Statement

This report uses the CMPSC 448 midterm project brief and the project's dataset partitions, experiment configurations, training histories, classification reports, prediction records, and RQ1–RQ4 summary files.

LLM assistance was used in prompt design, code development, and report preparation. The author is responsible for verifying, understanding, and explaining the submitted code and results.